

GAAO 1.0 (General Adaptive Agent Ontology), a unified event-sourced ontology for modeling and steering adaptive agents through time, integrating semantic topology, constraint systems, condition models, evidential reasoning, transformation dynamics, and recursive adaptation loops

Robert Swaab

December 9, 2025

Abstract

We present GAAO 1.0 (General Adaptive Agent Ontology), a unified event-sourced ontology for modeling and steering adaptive agents through time, integrating semantic topology, constraint systems, condition models, evidential reasoning, transformation dynamics, and recursive adaptation loops. GAAO formalizes a general-purpose adaptive agent as the tuple $A = (E, C, K, X, R, P, \Omega, I, L)$, where the agent is defined by an event-sourced ledger of behaviour, a hierarchical semantic topology, an explicit constraint fabric, a structured condition space, an evidential graph, a transformation layer capturing progress metrics and outcome effects, an adaptive reasoning engine, and a recursive adaptation loop governing temporal evolution of state. Unlike existing paradigms such as BDI, ACT-R, reinforcement learning, MAPE-K, or event-sourced architectures, GAAO unifies semantics, constraints, conditions, evidence, transformation dynamics, and recursive feedback into a single ontology for modeling and steering adaptive behaviour over time.

Introduction

Adaptive agents—biological, artificial, organisational, or hybrid—must continuously interpret conditions, choose actions, accumulate evidence, and adjust behaviour across time. Most existing architectures focus on a subset of this problem:

- BDI: beliefs, desires, intentions and plan choice,
- ACT-R: human cognitive processes via production rules and memory systems,
- reinforcement learning (RL): reward maximisation in Markovian environments,
- MAPE-K: monitor–analyse–plan–execute over a knowledge base,
- event-sourced architectures: state reconstruction from event streams.

What is missing is a single ontology that simultaneously:

1. treats behaviour as an event-sourced ledger over time,
2. situates behaviour in a semantic topology of containers,
3. represents constraints as explicit commitments with evaluation functions,
4. models conditions as a structured space of dimensions and models,
5. treats evidence, deltas, drift, and trajectories as first-class objects,
6. captures progress and outcomes as transformation dynamics,

7. and closes the loop with a recursive adaptation operator over agent state.

GAAO is proposed as such an ontology. It is not a single algorithm or implementation, but a formal structure from which many architectures (including human life systems, software agents, organisational agents, and hybrid human–AI systems) can be instantiated.

In what follows we:

- formally define the GAAO tuple and its layers,
- provide a mathematical specification,
- compare GAAO dimensionally with existing architectures (BDI, ACT-R, RL, MAPE-K, event-sourced systems),
- and synthesise the uniqueness of GAAO as a unifying ontology for adaptive agents.

Symbol Table

Table 1 summarises the primary symbols used in GAAO.

Symbol table for GAAO.

Symbol	Meaning
T	Time domain
A	Attribute space
M	Engagement mode set
C	Semantic containers
$\square$	Container-type set
K	Constraints
$\square(K)$	Power set of constraints K
X_d	Condition dimensions
X_m	Condition models
X	Condition space ($X_d \cup X_m$)
R	Evidence records
P	Progress records
Ω	Outcome records
D	Deviation space
$\mathcal{A}$	Metric space (for progress)
$\square_r$	Trajectory extraction operator
Drift	Drift extraction operator
Φ	Pattern space (drift and trajectories)
Ψ	Deviation-classification space
L	Recursive adaptation loop schema
$\mathbf{I}$	State-transition operator induced by L

Formal Definition of the General Adaptive Agent Ontology

A General Adaptive Agent in GAAO is defined as:

$$A = (E, C, K, X, R, P, \Omega, I, L),$$

where:

- Event Ledger Layer,

- Semantic Topology Layer,
- Constraint Fabric,
- Condition Space Layer,
- Evidential Graph Layer,
- Progress Records (part of transformation layer),
- Outcome Records (part of transformation layer),
- Adaptive Reasoning Engine,
- Recursive Adaptation Loop (schema and induced operator).

We now define each component.

Event Ledger Layer

Let:

T (time domain), A (attribute space), M (engagement mode set).

An event is defined as:

$$e = (t_s, t_e, \lambda_c, \lambda_m, \pi, \alpha, \sigma, \omega, \delta, \kappa),$$

where:

$$\begin{aligned} & t_s, t_e \in T \text{ \textit{\text{start and end times}}}, \lambda_c \in C \text{ \textit{\text{semantic container identifier}}}, \lambda_m \in M \text{ \textit{\text{engagement mode}}}, \\ & \pi, \alpha \in A \text{ \textit{\text{planned and actual attributes}}}, \sigma \in X \text{ \textit{\text{condition snapshot}}}, \omega \in \Omega \text{ \textit{\text{outcome record}}}, \delta \in D \text{ \textit{\text{deviation signal}}}, \\ & \kappa \subseteq K \text{ \textit{\text{applicable constraints}}}. \end{aligned}$$

The Event Ledger is the set:

$$E = \{e_1, e_2, \dots\}.$$

As a type, we can write:

$$E \subseteq T \times T \times C \times M \times A \times A \times X \times \Omega \times D \times \square(K),$$

where $\square(K)$ is the power set of K .

Semantic Topology Layer

Let $\square$ be the set of container types.

A semantic container is:

$$c = (id, \tau, parent, \square, H_e, H_\omega, H_p, H_\phi),$$

where:

- id is a unique container identifier,
- $\tau \in \square$ is the container type,
- $parent \in C \cup \{\emptyset\}$ is the unique parent container or null (root),
- $\square$ is a container state vector,
- H_e is event history attached to the container,
- H_ω is outcome history,
- H_p is progress history,

- H_ϕ is drift history (sequence of drift signals ϕ).

The semantic topology is a rooted tree:

$$G = (C, \text{parent}),$$

with standard tree properties (one parent, acyclic, connected at the root).

Constraint Fabric

A constraint is:

$$k = (id, \iota, \theta, \mu, W, L_C, \gamma, H_k),$$

where:

- ι is an intention descriptor,
- θ is an obligation specification,
- μ is a measurement mode,
- $W \subseteq T$ is an activation window,
- $L_C \subseteq C$ is the set of bound containers,
- γ is an evaluation function,
- H_k is the constraint's adjustment history.

For a given constraint $k \in K$, the evaluation function is:

$$\gamma_k : (E, X, C) \rightarrow [0, 1] \cup \{\text{fulfilled, violated}\}.$$

Transformation Layer: Progress and Outcomes

Let $\mathcal{A}$ be the metric space for progress measurement.

A progress record is:

$$p = (c, \text{metric}, v, d, e, t),$$

where:

- $c \in C$ is the container in which progress occurs,
- $\text{metric} \in \mathcal{A}$ is the progress metric,
- v is the magnitude,
- $d \in \{-1, 0, 1\}$ is the direction (away from, neutral, toward),
- $e \in E$ is the originating event,
- $t \in T$ is the timestamp.

An outcome record is:

$$\omega = (i, x, s, \delta),$$

where:

- i captures internal effects,
- x captures external effects,
- s is a state-transition marker,
- $\delta \in D$ is a deviation classification.

Condition Space Layer

Let:

X_d be the set of condition dimensions, X_m be the set of condition models, $X = X_d \cup X_m$.

A condition dimension is:

$$\xi = (name, type, value, t, source, conf, exp),$$

capturing an atomic contextual or state-like quantity.

A condition model is:

$$M = (name, type, inputs, rules, update, confidence, version),$$

where:

- $inputs \subseteq X_d$,
- $rules$ encodes interpretive logic,
- $update : (X, R) \rightarrow X$ defines model evolution,
- $confidence$ is a model-level confidence score,
- $version$ is a revision identifier.

The condition profile at time t is:

$$X_t \subseteq X.$$

Evidential Graph Layer

Let D be the deviation space.

An evidence record is:

$$r = (id, type, t, c, k, e, raw, derived, conf, src),$$

where $type$ may be one of: {EventEvidence, DeltaEvidence, ProgressEvidence, OutcomeEvidence, DriftSignal, TrajectorySignal}.

A delta record is:

$$\delta = f_\delta(\pi, \alpha) \in D,$$

computed from planned vs. actual attributes.

A drift signal is:

$$\phi = (type, magnitude, recurrence, link, t_1, t_n),$$

and a trajectory signal is:

$$\tau = (pattern, container, direction, confidence, window).$$

In addition to the delta operator f_δ , GAAO defines evidential extraction operators:

$$\square_r : R \rightarrow \{\tau_1, \dots, \tau_m\},$$

$$\text{Drift} : R \rightarrow \{\phi_1, \dots, \phi_n\},$$

mapping evidence records into recognised trajectory and drift-signal sets.

Adaptive Reasoning Engine

The adaptive reasoning engine is:

$$I : (X, E, K, C, R) \rightarrow \{\Pi, \Delta, S, Y\},$$

- $\mathcal{Q}$ — outcome records,
- $\mathcal{D}$ — deviation space,
- $\mathcal{A}$ — metric space.

Events inhabit:

$$E \subseteq T \times T \times C \times M \times A \times A \times X \times \mathcal{Q} \times \mathcal{D} \times \square(K).$$

The condition profile:

$$X_t \subseteq X.$$

Semantic topology:

$$G = (C, \text{parent}), \quad \text{parent} : C \rightarrow C \cup \{\emptyset\}.$$

Constraint evaluation:

$$\gamma_k : (E, X, C) \rightarrow [0, 1] \cup \{\text{fulfilled, violated}\}.$$

Condition dynamics:

$$\text{update}_M : (X, R) \rightarrow X, \quad \text{update}_X : (X, R) \rightarrow X.$$

Evidential operators:

$$f_\delta : (\pi, \alpha) \rightarrow D, \quad \square_r : R \rightarrow \{\tau_1, \dots, \tau_m\}, \quad \text{Drift} : R \rightarrow \{\phi_1, \dots, \phi_n\}.$$

Adaptive reasoning:

$$I : (X, E, K, C, R) \rightarrow \{\Pi, \Delta, S, Y\}, \quad I_\Delta : D \rightarrow \Psi.$$

Recursive adaptation:

$$\text{state}_t = (K_t, C_t, X_t, E_t, R_t), \quad \text{state}_{t+1} = \mathbf{I}(\text{state}_t).$$

Comparison to Existing Agent Architectures

We sketch dimensional comparisons; these can be expanded in a longer version.

BDI (Belief–Desire–Intention)

BDI focuses on beliefs (world knowledge), desires (goals), intentions (committed goals), and plans. In GAAO terms:

- Beliefs correspond to aspects of X and R ,
- Desires correspond to intention descriptors ι in K ,
- Intentions correspond to active constraints over windows W ,
- Plans correspond to outputs of I_{plan} and I_{sim} .

BDI captures a subset of GAAO’s concerns: primarily goal/commitment management and deliberative planning. GAAO additionally formalises evidential truth, delta and drift, semantic topology, and condition models as first-class components of the ontology.

ACT-R

ACT-R is a cognitive architecture for human cognition built on production rules and multiple memory systems. GAAO differs by:

- using an explicit event ledger E ,

- modelling semantics via containers C and types $\square$,
- treating constraints K as commitments with evaluation functions,
- modelling conditions via X_d, X_m ,
- using an evidential graph R with drift and trajectories.

ACT-R provides a rich internal cognitive model, but does not attempt to be an event-sourced, constraint-governed, semantics-explicit ontology as GAAO does.

Reinforcement Learning (RL)

Reinforcement learning formalises:

- state space,
- action space,
- reward function,
- policy and value functions.

In contrast:

- RL optimises cumulative reward; GAAO maintains constraint alignment and adaptive behaviour under multiple signals.
- RL states are Markovian vectors; GAAO states include condition profiles X_t and container states $\square$.
- RL rewards are scalar; GAAO outcomes ω are multi-signal internal and external transformations.
- GAAO explicitly models semantics, constraints, evidence, and multi-scale adaptation.

MAPE-K

MAPE-K (monitor–analyse–plan–execute over knowledge) is a loop pattern for self-adaptive systems. GAAO can be viewed as a granular refinement:

- Monitor $\rightarrow$ Event Ledger + Evidential Graph (E, R) ,
- Analyse $\rightarrow I_{\text{int}}, I_{\text{pat}}, I_{\text{A}}$ over (X, R) ,
- Plan $\rightarrow I_{\text{plan}}$,
- Execute $\rightarrow$ event realisation and updates to E ,
- Knowledge $\rightarrow (C, X, R)$ plus constraint fabric K .

Event-Sourced Architectures

Event-sourced systems store events as canonical truth and reconstruct state from event streams. GAAO adopts this backbone but extends it with:

- a semantic topology C ,
- an explicit constraint fabric K ,
- condition space X ,
- evidential graph R ,

- transformation layer $(P, \mathcal{Q})$,
- adaptive reasoning I ,
- recursive adaptation loop $\mathbf{I}$.

Synthesis of Uniqueness

We summarise the structural differences of GAAO without overclaiming.

Cross-Domain Integration

GAAO integrates:

- temporal event sourcing (Event Ledger E),
- hierarchical semantic structure (Semantic Topology C),
- formal constraint algebra (Constraint Fabric K),
- condition-space modelling (Condition Space X),
- evidential truth engine (Evidential Graph R with deltas, drift, trajectories),
- transformation modelling (Progress P and Outcomes $\mathcal{Q}$),
- adaptive reasoning (engine I),
- recursive adaptation loop $(L, \mathbf{I})$.

Existing frameworks address subsets of this list. GAAO's contribution is a unified ontology spanning all of them.

Delta and Drift as First-Class Constructs

GAAO treats:

- $\delta = f_{\delta}(\pi, \alpha)$ (delta),
- drift signals ϕ ,
- trajectory signals τ ,

as first-class objects tightly coupled with constraints, containers, and trajectories. This provides a principled way to reason about misalignment between intended and actual behaviour across time.

Semantics, Evidence, Constraints Co-Defined

GAAO co-defines:

- where things happen (semantic containers C),
- what is supposed to happen (constraints K),
- what actually happens (events E and evidence R),
- what changes (transformation layer $(P, \mathcal{Q})$),
- under what conditions it happens (condition space X),
- how behaviour is interpreted and adjusted (engine I and loop $\mathbf{I}$).

This tight coupling of semantic structure, evidence, and constraints is not typical in existing agent or control architectures.

Positioning Statement

A cautious but accurate positioning is:

To our knowledge, GAAO offers a novel integration of event-sourced behaviour, hierarchical semantic structure, constraint governance, condition-space modelling, evidential reasoning, and multi-scale adaptation within a single agent ontology. Existing frameworks such as BDI, ACT-R, RL, MAPE-K, and event-sourced system designs each address subsets of these concerns. GAAO is intended as a unifying ontology that can host and interoperate with those approaches while providing an explicit temporal–semantic–constraint–evidential backbone.

Conclusion

We have introduced GAAO and provided a formal specification of its core components. By defining adaptive agents as tuples over event ledgers, semantic topologies, constraint fabrics, condition spaces, evidential graphs, transformation layers, adaptive reasoning engines, and recursive adaptation loops, GAAO offers a general ontology for modeling and steering adaptive behaviour over time.

Future work includes:

- instantiating specific architectures derived from GAAO (e.g. cognitive agents, life-architecture systems, organisational agents),
- developing formal proofs about stability, convergence, or safety properties under specific instantiations of **I**,
- empirical evaluation of GAAO-based architectures in real-world domains.